

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO5: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Analytical Problem-Solving Assessment

Duration: 1hr

1. A warehouse robot must determine the shortest path to transport goods while avoiding obstacles. Apply Dynamic Programming to develop an optimal policy and explain how value iteration can improve navigation efficiency.

1. Introduction

A warehouse robot is required to transport goods from a starting location to a target destination while avoiding obstacles and minimizing the total distance or movement cost.

This problem can be modeled as a Markov Decision Process (MDP) and solved using Dynamic Programming (DP). One suitable DP technique is Value Iteration, which repeatedly calculates the expected value of each state and determines the best action.

The main objective is:

Find the shortest and safest path from the robot's starting position to the destination while avoiding obstacles.

2. Problem Representation

Consider the following warehouse grid:

1	2	3	4	5
+-----+-----+-----+-----+-----+				
A	Start		X	

```

+-----+-----+-----+-----+
B |   |   | X |   |   | X |   |
+-----+-----+-----+-----+
C |   |   |   |   |   |   |   |
+-----+-----+-----+-----+
D | X |   |   | X |   |   |   |
+-----+-----+-----+-----+
E |   |   |   |   |   | Goal |
+-----+-----+-----+-----+

```

Here:

- **Start** = initial position of the robot
- **Goal** = location where goods must be delivered
- **X** = obstacle
- Empty cells = accessible warehouse locations

The robot must find an optimal path from **Start** → **Goal**.

3. MDP Formulation

The warehouse navigation problem can be represented using five important components:

$MDP = (S, A, P, R, \gamma)$

Where:

- SS = Set of states
- AA = Set of actions
- PP = Transition probabilities
- RR = Reward function
- γ = Discount factor

4. States

A **state** represents the current position of the robot in the warehouse.

For example:

$S = \{(1,1), (1,2), (1,3), \dots, (5,5)\}$

Each grid cell represents one state.

For example:

(1,1) → Robot at first row, first column
(2,3) → Robot at second row, third column
(5,5) → Goal state

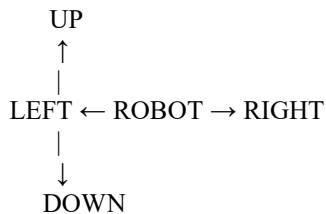
Obstacle cells are not considered valid navigation states.

5. Actions

At every accessible state, the robot can perform actions such as:

- **Up**
- **Down**
- **Left**
- **Right**

For example:



If an action would move the robot into an obstacle or outside the warehouse, that action is considered invalid or given a large negative reward.

6. Transition Model

The transition model determines the next state after an action.

For example:

If the robot is at:

(2,3)(2,3)

and chooses:

RightRight

then it moves to:

(2,4)(2,4)

if that cell is free.

For a deterministic warehouse environment:

$$P(s' | s, a) = 1 \quad P(s' | s, a) = 1$$

for the intended next state.

If the robot has movement uncertainty, the transition probabilities can instead represent the possibility of moving to neighboring cells.

7. Reward Function

The reward function encourages the robot to reach the destination quickly while avoiding obstacles.

A suitable reward structure is:

Situation	Reward
Reach goal	+100
Normal movement	-1
Move toward efficient path	-1
Hit obstacle	-50
Invalid movement	-20
Long/inefficient route	Additional negative cost

For example:

$R(s,a,s') = \begin{cases} +100 & \text{if goal is reached} \\ -50 & \text{if obstacle is hit} \\ -1 & \text{for normal movement} \end{cases}$

The **-1 movement cost** encourages the robot to reach the goal using fewer steps.

8. Dynamic Programming

Dynamic Programming solves the navigation problem by breaking it into smaller subproblems.

Instead of calculating every possible path from the beginning, DP stores the value of previously evaluated states and reuses them.

The robot determines:

"How good is it to be in this particular location?"

This value is represented by:

$V(s)$

where $V(s)$ is the expected long-term reward from state s .

9. Bellman Optimality Equation

The fundamental equation used by value iteration is:

$$V^*(s) = \max_a [R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^*(s')] \quad V^*(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^*(s') \right]$$

For a deterministic environment, this becomes:

$$V^*(s) = \max_a [R(s,a) + \gamma V^*(s')] \quad V^*(s) = \max_a [R(s,a) + \gamma V^*(s')]$$

Where:

- $V^*(s)$ = optimal value of state s

- a = possible action
- $R(s,a)$ = immediate reward
- γ = discount factor
- s' = next state
- $P(s' | s,a)$ = transition probability

10. Value Iteration

Value Iteration is a Dynamic Programming algorithm used to calculate the optimal value of every state.

Initially, values are assigned to all states.

For example:

Goal = 100

All other states = 0

Obstacles = $-\infty$

Then the algorithm repeatedly updates the values using the Bellman equation.

Iteration process

Initialize $V(s)$



Evaluate all possible actions



Calculate new state values



Select maximum value



Update $V(s)$



Repeat until convergence



Extract optimal policy

11. Value Iteration Example

Suppose the robot is near the goal.

Consider:

```
+-----+-----+-----+
|   |   |   |   |
+-----+-----+-----+
|   | R |   |   |
+-----+-----+-----+
|   |   | Goal |
+-----+-----+-----+
```

The robot is represented by **R**.

The algorithm evaluates:

- Move Up

P. Sahith Sai
192425029
BTech AIML

- Move Down
- Move Left
- Move Right

If moving right eventually leads to the goal with fewer steps, its value becomes higher.

For example:

Goal = 100

Nearby state = 99

Next state = 98

Next state = 97

...

The high values propagate backward from the goal to other states.

This allows the robot to identify the direction that leads toward the goal.

12. Value Table

After several iterations, the warehouse may produce a value table similar to:

```
+-----+-----+-----+-----+
| 92 | 93 | 94 | XX |
+-----+-----+-----+-----+
| 91 | XX | 95 | XX |
+-----+-----+-----+-----+
| 90 | 89 | 96 | 97 |
+-----+-----+-----+-----+
| XX | 88 | XX | 98 |
+-----+-----+-----+-----+
| 86 | 87 | 88 | 100 |
+-----+-----+-----+-----+
```

XX represents obstacles.

The robot chooses actions that move toward states with higher values.

13. Extracting the Optimal Policy

Once the value function converges, an **optimal policy** can be obtained.

The policy is:

$$\pi^*(s) = \arg \max_a [R(s,a) + \gamma V^*(s')] \quad \pi^*(s) = \arg \max_a [R(s,a) + \gamma V^*(s')]$$

The policy tells the robot which action to take from every state.

For example:

```
↑ ↑ → X
↑ X ↓ X
↑ ← → ↓
```

This means that at every location, the robot knows the best direction to take.

14. Shortest Path

After obtaining the optimal policy, the robot follows it from the starting state.

Example:

START
↓
↓
→
→
↓
↓
→
GOAL

The robot avoids obstacle cells and follows the route with the highest expected cumulative reward.

Because every normal movement has a negative cost, the algorithm naturally prefers a **shorter path**.

15. How Value Iteration Improves Navigation Efficiency

Value Iteration improves warehouse navigation in several ways.

1. Finds optimal paths

It evaluates all possible actions and selects the action with the highest long-term value.

2. Avoids obstacles

Obstacle states can be assigned large negative values, discouraging the robot from entering them.

3. Reduces unnecessary movement

Giving every movement a small negative reward encourages the robot to reach the destination using fewer steps.

4. Provides a policy for every state

Instead of calculating a path only from one starting position, Value Iteration can calculate the best action for every reachable state.

5. Handles changing conditions

If obstacles or rewards change, the value function can be recalculated to generate a new policy.

6. Reduces computational repetition

Dynamic Programming stores previously calculated state values instead of repeatedly solving the same subproblems.

16. Pseudocode

Initialize $V(s) = 0$ for all states

Set $V(\text{goal}) = 100$

Repeat:

$\Delta = 0$

For every non-obstacle state s :

old_value = $V(s)$

For every possible action a :

Calculate:

$R(s,a) + \gamma V(s')$

$V(s) = \text{maximum calculated value}$

$\Delta = \text{maximum}(\Delta, |\text{old_value} - V(s)|)$

Until $\Delta < \text{threshold}$

For every state:

Select action with maximum value

Return optimal policy

17. Python Implementation

A simple implementation can be used to demonstrate the concept:

```
import numpy as np
grid = [
    ['S', '.', '.', '#', '.'],
    ['.', '#', '.', '#', '.'],
    ['.', '.', '.', '.', '.'],
    ['#', '.', '#', '.', '.'],
    ['.', '.', '.', '.', 'G']
]
rows = len(grid)cols = len(grid[0])
gamma = 0.9threshold = 0.001
V = np.zeros((rows, cols))
for i in range(rows):
    for j in range(cols):
        if grid[i][j] == 'G':
            V[i][j] = 100
        elif grid[i][j] == '#':
            V[i][j] = -100
actions = {
```


P. Sahith Sai

192425029

BTech AIML

```
'U': (-1, 0),  
'D': (1, 0),  
'L': (0, -1),  
'R': (0, 1)
```

```
}
```

```
while True:
```

```
    new_V = V.copy()
```

```
    for i in range(rows):
```

```
        for j in range(cols):
```

```
            if grid[i][j] in ['#', 'G']:
```

```
                continue
```

```
            values = []
```

```
            for action, (di, dj) in actions.items():
```

```
                ni = i + di
```

```
                nj = j + dj
```

```
                if ni < 0 or ni >= rows or nj < 0 or nj >= cols:
```

```
                    values.append(-20 + gamma * V[i][j])
```

```
                elif grid[ni][nj] == '#':
```

```
                    values.append(-50 + gamma * V[i][j])
```

```
                elif grid[ni][nj] == 'G':
```

```
                    values.append(100)
```

```
                else:
```

```
                    values.append(-1 + gamma * V[ni][nj])
```

```
            new_V[i][j] = max(values)
```

```
    if np.max(np.abs(new_V - V)) < threshold:
```

```
        V = new_V
```

```
        break
```

```
    V = new_V
```

```
    policy = []
```

```
    for i in range(rows):
```

```
        row = []
```

```
        for j in range(cols):
```

```
            if grid[i][j] in ['#', 'G']:
```

```
                row.append(grid[i][j])
```

```
            continue
```

```
        best_action = None
```

```
        best_value = -float('inf')
```

```
        for action, (di, dj) in actions.items():
```

```
            ni = i + di
```

```
            nj = j + dj
```

```
            if ni < 0 or ni >= rows or nj < 0 or nj >= cols:
```

```
        value = -20 + gamma * V[i][j]
    elif grid[ni][nj] == '#':
        value = -50 + gamma * V[i][j]
    elif grid[ni][nj] == 'G':
        value = 100
    else:
        value = -1 + gamma * V[ni][nj]

    if value > best_value:
        best_value = value
        best_action = action

    row.append(best_action)

    policy.append(row)
print("Value Function:")print(np.round(V, 2))
print("\nOptimal Policy:")
for row in policy:
    print(row)
```

18. Expected Output

The output will contain a **value function** similar to:

Value Function:

```
[[... ... ... ...]
 [... ... ... ...]
 [... ... ... ...]
 [... ... ... ...]
 [... ... ... 100]]
```

And an optimal policy such as:

Optimal Policy:

```
['D', 'R', 'D', '#', 'D']
['D', '#', 'D', '#', 'D']
['R', 'R', 'R', 'R', 'D']
['#', 'D', '#', 'D', 'D']
['R', 'R', 'R', 'R', 'G']
```

The exact values may vary depending on the reward structure and discount factor.

19. Advantages of Dynamic Programming

Dynamic Programming is suitable for this problem because:

- It provides an **optimal policy** when the MDP model is known.
- It avoids recalculating the same subproblems.
- It can handle obstacles through the transition and reward models.
- It considers **future rewards**, not just the immediate movement.
- It can generate a policy for the entire warehouse.
- It provides a systematic approach to shortest-path planning.

20. Limitations

There are also some limitations:

1. **Large state space:** A very large warehouse can result in thousands or millions of states.
2. **Known environment:** Traditional DP assumes that the transition and reward models are known.
3. **Computation cost:** Repeated value updates can be expensive for very large environments.
4. **Dynamic obstacles:** If obstacles continuously move, the value function may need to be updated frequently.
5. **Real-world uncertainty:** Sensors may be noisy and the robot may not always move exactly as intended.

21. Final Answer / Conclusion

The warehouse robot navigation problem can be modeled as an MDP in which each warehouse location represents a state, movement directions represent actions, and rewards represent the cost or benefit of each movement. Dynamic Programming, particularly Value Iteration, repeatedly applies the Bellman optimality equation to calculate the value of each state.

Once the values converge, the robot selects the action with the highest expected value from each state, producing an optimal navigation policy. Assigning negative rewards to movement and obstacles while providing a high reward for reaching the goal encourages the robot to find a short, safe, and efficient path.

Therefore, Value Iteration improves warehouse navigation efficiency by systematically evaluating future rewards, avoiding obstacles, minimizing unnecessary movements, and generating an optimal policy for reaching the destination.

Code of Conduct:

I P. Sahith Sai (192425029) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P. Sahith Sai

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	25	Demonstrates deep understanding of concepts with complete clarity	Explains concepts correctly with minor gaps	Partial understanding with errors or omissions	Unable to explain basic concepts
Application	25	Effectively applies concepts to new situations; solves problems logically	Applies concepts with minor errors	Limited ability to apply concepts; requires guidance	Unable to apply concepts effectively
Analytical Thinking	20	Critically analyzes scenarios with strong justification and reasoning	Provides reasonable analysis with some justification	Superficial analysis with weak reasoning	No analytical reasoning demonstrated
Communication	15	Communicates ideas clearly, confidently, and with appropriate terminology	Communicates clearly but with minor hesitation	Communication lacks clarity or structure	Unable to communicate ideas effectively
Response to Questions	15	Responds accurately and confidently, extending answers with depth	Responds correctly but with limited depth	Responds partially; requires prompting	Unable to respond to questions effectively